

RakshaNet: Detailed Project Report

Project Name: RakshaNet

Repository: <https://github.com/Sainiamritansh/RakshaNet>

Developer: Amritansh Saini (Sainiamritansh)

Report Generated: April 2026

Executive Summary

RakshaNet is a geo-based emergency response application designed to bridge the critical gap between emergency situations and immediate assistance. The application serves dual purposes: addressing physical emergencies (accidents, medical crises, natural disasters) and digital scam situations (financial fraud, cybercrime). By leveraging real-time location sharing, guided emergency protocols, and instant contact coordination, RakshaNet aims to reduce response times and minimize damage during high-stress situations where every second counts.

The application targets mobile users who require quick, reliable access to emergency services and trusted contacts during critical moments.

1. Project Overview

1.1 Vision & Purpose

The core vision of RakshaNet is to create a seamless emergency response ecosystem that empowers users to:

- Immediately alert trusted contacts about emergencies with a single SOS action
- Share real-time location data for faster rescue operations
- Receive guided emergency response steps tailored to the specific situation
- Coordinate with emergency contacts without delays
- Handle both physical emergencies and digital scams through a unified platform

1.2 Problem Statement

Physical Emergencies:

- Users need seconds to contact help during critical situations
- Traditional emergency response involves multiple app switches (maps, calling, messaging)
- Location data is not automatically shared with trusted contacts

- Emergency responders lack real-time victim location information

Digital Scams:

- Users affected by online fraud lack immediate guidance
- No unified platform to alert contacts about scams in progress
- Limited access to emergency resources and hotline numbers
- Confusion about proper response procedures increases damage

1.3 Key Objectives

1. **Single-Action SOS:** Enable users to trigger comprehensive emergency alerts with one action
 2. **Real-Time Geolocation:** Automatically capture and share user location with emergency contacts
 3. **Guided Response:** Provide contextual emergency guidance based on situation type
 4. **Contact Coordination:** Facilitate quick communication with pre-registered trusted contacts
 5. **Dual Emergency Support:** Address both physical and digital emergencies
 6. **Cross-Platform Availability:** Support both Android and iOS platforms
-

2. Technical Architecture

2.1 Technology Stack

Frontend/Mobile Development

- **Framework:** React Native (Expo-based)
- **Language:** JavaScript/JSX
- **Build System:** Metro Bundler (with custom configuration)
- **Package Manager:** npm

Backend Infrastructure

- **Architecture Type:** Microservices (service-oriented)
- **API Layer:** Service modules in `/services` directory
- **Context Management:** React Context API for state management

Android-Specific Components

- **Native Android:** Java and Kotlin support
- **Build Tool:** Gradle-based Android module
- **EAS Build:** Expo Application Services for cloud builds

DevOps & Deployment

- **EAS Configuration:** eas.json for Expo deployment
- **Environment Management:** .env file for configuration
- **Version Control:** Git-based version management

2.2 Project Structure

```
RakshaNet/  
├── Components/           # Reusable React components  
├── Pages/               # Screen/page components  
├── android/             # Native Android modules  
├── assets/              # Images, icons, fonts  
├── context/             # React Context API state management  
├── services/            # Backend service integration  
├── src/                 # Source code (additional)  
├── App.jsx              # Root application component  
├── index.js             # Application entry point  
├── app.json             # Expo configuration  
├── eas.json             # Expo deployment configuration  
├── metro.config.js      # Metro bundler configuration  
├── package.json         # Dependencies  
├── .env                 # Environment variables  
├── .gitignore           # Git exclusions  
└── architecture-graph.html # System architecture visualization
```

2.3 Language Distribution

- **JavaScript:** 75.0% (primary development language)
- **HTML:** 15.1% (UI markup and web components)
- **Java:** 8.1% (Android native modules)
- **Kotlin:** 1.8% (Modern Android development)

2.4 Architecture Diagram

The project includes an `architecture-graph.html` file that visualizes the system components and their relationships, providing a comprehensive view of how different modules interact.

3. Core Features & Functionality

3.1 SOS Emergency Alert System

Feature: Single-button emergency activation

- **Trigger Mechanism:** One-tap SOS button on main interface
- **Functionality:**
 - Immediate alert to all pre-registered emergency contacts
 - Automatic location capture and broadcasting
 - Emergency status flag for routing through help systems
 - Audio/visual alert to nearby people

3.2 Real-Time Geolocation Sharing

Feature: Continuous location tracking and sharing

- **Components:**
 - GPS-based location acquisition
 - Background location service
 - Real-time location updates to contacts
 - Maps integration for location visualization
 - Location history logging

3.3 Guided Emergency Response Steps

Feature: Contextual emergency procedures

- **Emergency Types Covered:**
 - Medical emergencies (heart attack, stroke, severe bleeding)
 - Accident/Trauma situations
 - Natural disasters (earthquakes, floods)
 - Digital scams (fraud, cybercrime)

- Assaults/Safety threats
- **Guidance Includes:**
 - Step-by-step instructions
 - Do's and don'ts for specific situations
 - First aid basics
 - When to call specific emergency services
 - Evidence preservation tips

3.4 Trusted Contact Management

Feature: Emergency contact coordination

- **Capabilities:**
 - Add and categorize trusted contacts
 - Preset emergency contacts for different situation types
 - Contact notification preferences (SMS, push notification, call)
 - Contact acknowledgment tracking
 - Backup contact assignment

3.5 Digital Scam Response Module

Feature: Specialized scam handling

- **Scam Types Addressed:**
 - Banking/Financial fraud
 - Online shopping scams
 - Phone/Email phishing
 - Investment fraud
 - Social engineering
- **Features:**
 - Scam identification guidance
 - Immediate action checklists
 - Contact information for cyber authorities
 - Evidence documentation tools
 - Recovery resources

3.6 Emergency Resources & Hotlines

Feature: Quick access to services

- **Resources Included:**
 - Emergency service numbers (police, ambulance, fire)
 - Regional hotline directories
 - Mental health crisis lines
 - Poison control centers
 - Child/domestic violence helplines
 - Cyber crime reporting portals
-

4. Key Components

4.1 React Components ([/Components](#))

Reusable UI components including:

- SOS button component
- Location map display
- Contact list component
- Emergency guide reader
- Status indicator
- Contact notification system

4.2 Pages ([/Pages](#))

Main application screens:

- Home/Dashboard screen
- SOS activation screen
- Emergency contacts management
- Guided emergency steps view
- Digital scam resources
- User profile settings
- Location sharing view

4.3 Services (`/services`)

Backend integration layer:

- Location services API client
- Contact notification service
- Emergency database queries
- User authentication service
- Geolocation service wrapper
- Hotline database access

4.4 Context Management (`/context`)

State management using React Context:

- Emergency status context
- User authentication context
- Location context
- Contacts context
- Settings context

4.5 Android Native Module (`/android`)

Platform-specific functionality:

- Native Android activity management
- Permission handling (location, contacts, SMS)
- Background service for location tracking
- SMS messaging integration
- Native notifications

5. Development Workflow

5.1 Setup & Installation

```
bash
```

```
# Clone repository
git clone https://github.com/Sainiamritansh/RakshaNet.git
cd RakshaNet

# Install dependencies
npm install

# Configure environment
cp .env.example .env
# Edit .env with API keys and configuration

# Development server
npm start
# or for Expo
expo start
```

5.2 Build & Deployment

For Android:

```
bash

# EAS Build
eas build --platform android

# Local build
cd android
./gradlew assembleRelease
```

For iOS (via EAS):

```
bash

eas build --platform ios
```

Development/Testing:

```
bash

expo start --dev-client
```

5.3 Configuration Files

- **app.json:** Expo application configuration (app name, version, icon)

- **eas.json:** EAS build settings for deployment
 - **metro.config.js:** Metro bundler configuration for custom asset handling
 - **.env:** Environment variables (API endpoints, API keys)
 - **package.json:** Dependency management and npm scripts
-

6. Development Status & Metrics

6.1 Repository Activity

- **Total Commits:** 15 commits
- **Contributors:** 1 (Amritansh Saini)
- **Release Status:** No releases published yet
- **Project Stage:** Active development

6.2 Community Engagement

- **Stars:** 0 (early-stage project)
- **Watchers:** 0
- **Forks:** 0
- **Issues:** 0 (no open issues reported)
- **Pull Requests:** 0

6.3 Code Quality Indicators

- **Language Distribution:**
 - JavaScript-dominant (75%) - good for React Native
 - HTML (15%) - component markup
 - Java/Kotlin (10%) - native Android modules
 - **Project Structure:** Well-organized with clear separation of concerns
 - **Configuration Management:** Proper use of environment variables and build configs
-

7. Key Technologies & Dependencies

7.1 Core Framework

- **React Native:** Cross-platform mobile development
- **Expo:** Managed React Native platform

7.2 Location & Maps

- Geolocation APIs (likely using Expo Location or native)
- Maps integration (likely Google Maps or similar)

7.3 Backend Integration

- REST API communication
- Likely database for storing user data, contacts, and emergencies

7.4 Notifications

- Push notifications (Expo Notifications)
- SMS integration (likely Twilio or similar)
- Local notifications

7.5 Authentication

- User authentication system (details in context)
 - Session management
-

8. Use Cases & Scenarios

8.1 Medical Emergency Scenario

User: Rohit suffers a heart attack at home

Flow:

1. Rohit taps SOS button (or voice command)
2. App activates Emergency Mode
3. Location automatically shared with registered contacts
4. Emergency guidance displayed (CPR instructions, recovery position)

5. SMS alerts sent to wife, son, and neighbor
6. Ambulance service contact information provided
7. Real-time location tracking initiated
8. Emergency contacts can see Rohit's location on map
9. First responders can access shared location data

8.2 Road Accident Scenario

User: Sarah involved in car accident on highway

Flow:

1. Sarah activates SOS (or crash detection auto-triggers)
2. App identifies accident emergency type
3. Displays accident response guidance
4. Shares location with emergency contacts and nearby hospital
5. Provides police hotline information
6. Enables video recording for documentation
7. Alerts traffic authorities via location data
8. Contacts nearby hospitals about incoming patient

8.3 Digital Scam Scenario

User: Amit receives phishing SMS about banking fraud

Flow:

1. Amit recognizes potential scam, opens RakshaNet
2. Selects "Digital Scam" emergency type
3. Chooses "Banking Fraud" category
4. Receives immediate action checklist:
 - Don't share additional information
 - Contact bank's customer care (hotline provided)
 - File cyber crime report
 - Document all communications
5. Access to cyber crime authorities' contact information
6. Can notify trusted contacts about situation
7. Receives recovery guidance and resources

8.4 Natural Disaster Scenario

User: Family in earthquake-affected region

Flow:

1. Multiple family members activate SOS
 2. App identifies earthquake emergency
 3. Provides earthquake safety instructions
 4. Shows designated safe zones and shelters nearby
 5. Enables group location sharing
 6. Provides disaster relief hotlines
 7. Logs multiple SOSs for authorities (helps with resource allocation)
-

9. Security & Privacy Considerations

9.1 Data Security

Potential Security Measures:

- End-to-end encryption for location sharing
- Encrypted transmission of emergency data
- Secure storage of contact information
- Authentication for accessing shared locations
- Regular data cleanup/deletion policies

9.2 Privacy Protection

Privacy Features:

- Granular permission requests (location, contacts, SMS)
- User control over contact visibility
- Location sharing only during emergencies (unless configured)
- Option to disable background tracking
- Data minimization principles

9.3 Compliance

Potential Compliance Considerations:

- GDPR compliance for European users
 - India-specific data protection requirements (location data)
 - Healthcare data standards for medical information
 - Emergency services regulations and protocols
 - SMS regulations (DND rules for India)
-

10. Strengths & Advantages

1. **Unified Emergency Solution:** Single app for multiple emergency types
 2. **Dual-Purpose Design:** Addresses both physical and digital emergencies
 3. **Real-Time Coordination:** Instant location and contact coordination
 4. **User-Centric Approach:** Designed for high-stress situations with simplified UI
 5. **Cross-Platform:** React Native enables iOS and Android support
 6. **Guided Response:** Contextual guidance reduces user panic and improves response quality
 7. **Community Features:** Trusted contact networks provide social safety net
 8. **Scalability:** Microservices architecture allows feature expansion
-

11. Areas for Enhancement

11.1 Technical Improvements

1. **Offline Functionality:** Work without internet connection using cached contacts/guides
2. **AI-Powered Detection:** Integrate crash detection, fall detection, or emergency keyword recognition
3. **Biometric Authentication:** Fingerprint/face recognition for quick access
4. **Voice Commands:** Hands-free SOS activation for accessibility
5. **Integration with Emergency Services:** Direct API integration with 911/emergency dispatch systems

11.2 Feature Enhancements

1. **Community Alerts:** Notify nearby users about emergencies in their area
2. **Insurance Integration:** Auto-notify insurance companies for accidents
3. **Medical History:** Quick access to user's medical information for responders

4. **Language Support:** Multi-language interface for diverse users
5. **Video Streaming:** Live video to trusted contacts or responders
6. **Blockchain Verification:** For financial scam documentation

11.3 User Experience

1. **Emergency Contact Drills:** Periodic testing of contact notification system
2. **Achievement Badges:** Gamification for completing safety profiles
3. **Community Safety Maps:** Aggregate, anonymized safety data visualization
4. **Integration with Fitness Trackers:** Heart rate monitoring for health emergencies
5. **Customizable Emergency Profiles:** Different responses for different emergency types

11.4 Accessibility

1. **Text-to-Speech:** Guidance reading for visually impaired users
 2. **High Contrast Modes:** For better visibility in emergencies
 3. **Simplified UI Options:** Minimal UI for elderly users
 4. **Multi-Language Support:** Regional language support
 5. **Deaf/Hard of Hearing Support:** Visual alerts and captions
-

12. Monetization & Business Model

12.1 Potential Revenue Streams

1. **Freemium Model:** Basic features free, premium features (advanced guides, priority support) paid
2. **Enterprise Licensing:** Schools, offices, factories for employee safety
3. **Insurance Partnerships:** Integration with insurance providers
4. **Emergency Service Integration:** Partnerships with ambulance/police services
5. **Advertising:** Safety-related products and services (non-intrusive)
6. **Premium Subscriptions:** Advanced features, family plans, enhanced support

12.2 Target Markets

1. **Individual Consumers:** General public concerned about personal safety
2. **Educational Institutions:** Schools and universities for student safety
3. **Corporate Sector:** Companies prioritizing employee safety

4. **Healthcare Facilities:** Hospitals for emergency notification
 5. **Government Agencies:** Public safety implementations
 6. **Senior Citizens:** Elderly care applications
-

13. Future Roadmap

Phase 1: Beta Release (Current)

- Core SOS functionality
- Contact management
- Basic emergency guides
- Location sharing

Phase 2: Version 1.0 Release

- Digital scam module enhancement
- Offline functionality
- Multiple language support
- Performance optimization

Phase 3: Advanced Features

- AI-powered emergency detection
- Integration with emergency services APIs
- Video streaming capability
- Community safety features
- Health monitoring integration

Phase 4: Ecosystem Expansion

- Web dashboard for emergency coordinators
 - Integration with IoT devices
 - Wearable app versions
 - Smart home automation (alert lights, sirens)
-

14. Testing & Quality Assurance

14.1 Testing Strategy

Unit Testing:

- Component-level tests for React components
- Service layer tests for location and notification functionality
- Context management tests

Integration Testing:

- End-to-end SOS flow testing
- Contact notification system testing
- Location sharing validation

User Acceptance Testing:

- Real emergency simulation drills
- User interface usability testing
- Cross-device compatibility testing (various Android/iOS versions)

Performance Testing:

- Location update frequency optimization
- Battery impact assessment
- Network latency handling
- Large contact list performance

14.2 Quality Metrics

- Code coverage target: >80%
 - Performance: Location update < 5 seconds
 - Notification delivery: 99%+ success rate
 - Crash-free sessions: >99.9%
-

15. Deployment & Infrastructure

15.1 Platform Support

- **Android:** Minimum API level 21+, target latest stable
- **iOS:** iOS 13.0+ support through Expo
- **Web:** Potential web dashboard for administrators

15.2 Cloud Infrastructure

Likely Components:

- Frontend: Expo hosting or Firebase hosting
- Backend: Cloud functions or containerized microservices
- Database: Firebase Realtime DB or traditional PostgreSQL
- APIs: RESTful services on cloud provider (AWS, GCP, Azure)
- Storage: Cloud storage for logs and user data
- CDN: For asset distribution

15.3 Scalability Considerations

- Microservices allow independent scaling
- Database sharding for user data
- Geographically distributed servers for location services
- Load balancing for API endpoints
- Caching strategy for frequently accessed data

16. Challenges & Risk Mitigation

16.1 Technical Challenges

Challenge	Risk Level	Mitigation
Battery drain from location tracking	High	Implement smart tracking intervals, geo-fencing
Network latency during emergencies	High	Offline mode, cached data, hybrid connectivity
Cross-platform compatibility	Medium	Comprehensive testing, Expo managed updates
Large user base scalability	Medium	Cloud infrastructure, database optimization

16.2 Business Challenges

Challenge	Risk Level	Mitigation
User acquisition/adoption	High	Partnerships with institutions, safety campaigns
Regulatory compliance	High	Legal consultation, compliance team
Data privacy concerns	High	Transparent privacy policy, regular audits
Competition from established services	Medium	Unique features, focus on underserved markets

16.3 Operational Challenges

Challenge	Risk Level	Mitigation
24/7 support needs	High	AI chatbots, community forums, on-call team
Data accuracy (location, contacts)	Medium	Regular verification, user feedback
Integration with emergency services	High	Government relations, standardized protocols

17. Conclusion

RakshaNet represents a promising solution to a critical real-world problem: enabling faster, more coordinated emergency response. By combining location sharing, guided emergency procedures, and trusted contact networks, the app addresses gaps in current emergency response systems.

The project demonstrates solid technical fundamentals with a well-organized React Native codebase, appropriate use of microservices architecture, and clear separation of concerns. The dual-focus approach (physical + digital emergencies) differentiates it from competitors and broadens its market appeal.

Key Takeaways

- 1. **Strong Problem Definition:** Addresses real gaps in emergency response
- 2. **Scalable Architecture:** React Native and microservices allow growth
- 3. **User-Centric Design:** Simplified UX critical for high-stress situations
- 4. **Multi-Emergency Support:** Versatile solution for diverse emergency types
- 5. **Growth Potential:** Multiple monetization paths and market segments

Recommendations for Success

1. **Focus on User Safety:** Prioritize reliability and accuracy over feature quantity
 2. **Build Partnerships:** Work with emergency services and institutions early
 3. **Community Building:** Create network effects through trusted contacts
 4. **Privacy-First Approach:** Build trust through transparent data handling
 5. **Continuous Testing:** Regular emergency drills and user feedback incorporation
 6. **Geographic Expansion:** Start with specific regions, then expand nationally
-

18. Appendix

A. Contact Information

Developer: Amritansh Saini

GitHub Profile: <https://github.com/Sainiamritansh>

Repository: <https://github.com/Sainiamritansh/RakshaNet>

B. Project Metrics

- **Repository Language:** JavaScript (75%), HTML (15%), Java (8%), Kotlin (2%)
- **Total Commits:** 15
- **Contributors:** 1
- **Last Activity:** Recent (March/April 2026)
- **Status:** Active Development

C. Related Technologies & Ecosystems

- **React Native Ecosystem:** Expo, React Navigation, Redux/Context
- **Geolocation:** Google Maps API, Mapbox, or native location services
- **Notifications:** Firebase Cloud Messaging, Twilio, custom SMS APIs
- **Cloud Providers:** AWS, Google Cloud, Azure, or Firebase
- **Version Control:** Git, GitHub Actions for CI/CD

D. Regulatory & Compliance Considerations

- **India:** GDPR applicability, MEITY data handling guidelines
- **Emergency Services:** Compatibility with emergency dispatch systems
- **Healthcare:** HIPAA compliance for medical emergency data

- **Telecommunications:** SMS regulations (DND compliance in India)
 - **Insurance:** Coordination with insurance industry standards
-

Report Version: 1.0

Last Updated: April 23, 2026

Status: Comprehensive Analysis of Active Development Project